

面向 AI 原生系统的智能运维

第 1 讲 | 课程导论与 AI 原生系统架构概论

从模型、数据与算力，到可运行、可观察、可治理的 AI 系统

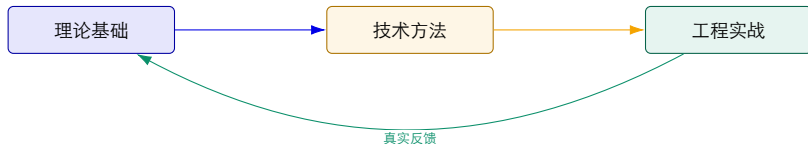
陈鹏飞 | 中山大学

本讲定位：先看清系统，再谈如何运维

课程对象 面向计算机及相关专业研究生，强调原理、工程与研究问题的连接。

课程场景 训练、推理、RAG、Agent、GPU 集群与企业级运维平台。

课程方法 校企双师协同：理论基础—技术方法—工程实战三段融合。



课程基本信息：34 学时，18 学时行业专家参与

34

总学时

18

行业专家负责学时

2

学分

17

教学周次

中山大学计算机学院

广州嘉为科技有限公司

研究方法课

学术前沿课

行业专家：张敏（CTO）、吴文豪（智能运维软件架构师）、周宗沛（智能运维产品总监）。

主讲人研究基础：从分布式系统可靠性走向 AI 原生运维

长期研究方向

- ▶ 分布式系统、云计算与操作系统；
- ▶ 计算机网络与无服务器计算；
- ▶ 软件可靠性、性能诊断与智能运维。

近期问题牵引

- ▶ 云原生系统的多模态故障检测与定位；
- ▶ 智算集群轻量全栈可观测性；
- ▶ LLM 驱动的根本原因分析与多智能体运维；
- ▶ AI/Agent 系统的故障注入与可靠性验证。

课程连接 课程沿着“系统如何运行—采集哪些数据—怎样诊断—如何处置—怎样验证恢复”展开。

课程助教



李俊艺

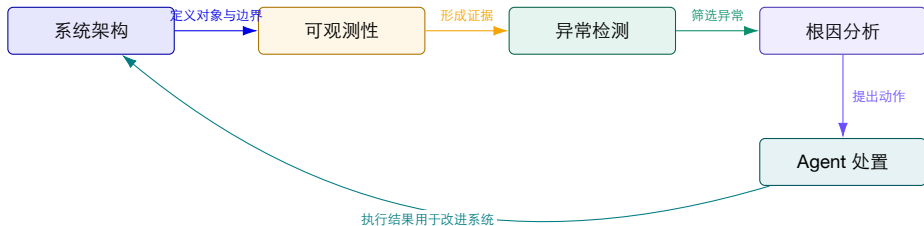
lijy727@mail2.sysu.edu.cn



余泽轩

yuzx23@mail2.sysu.edu.cn

课程内容为什么按这条顺序展开？

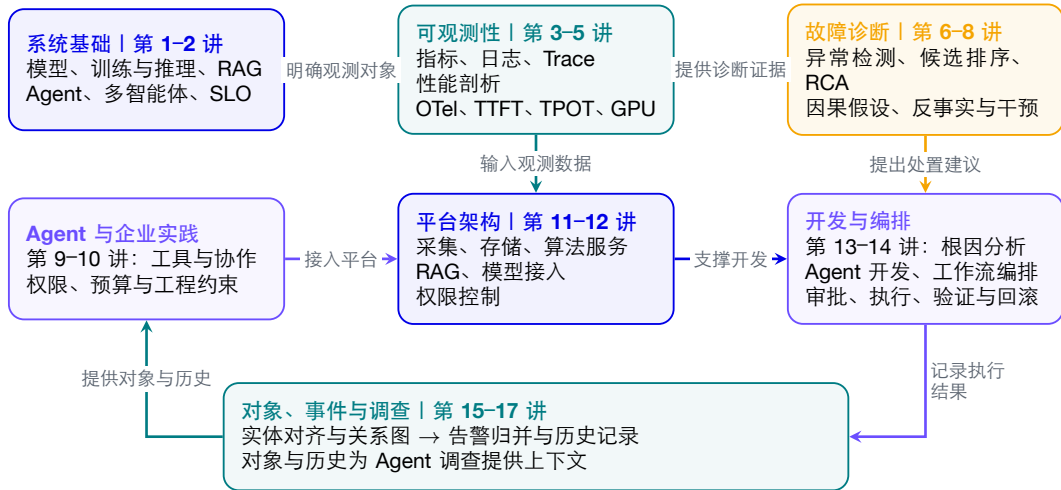


没有架构 不知道应该观察哪些对象。

没有证据 模型只能生成听起来合理的猜测。

没有验证 自动处置无法安全进入生产。

课程知识图谱：系统、证据、诊断与行动



为什么要开这门课？

系统正在变得更“智能”

- ▶ 模型成为需要单独发布、监控和回滚的核心组件；
- ▶ 数据、知识、工具和人共同参与一次请求；
- ▶ GPU、网络、存储和模型服务形成新的基础设施栈。

运维正在变得更“困难”

- ▶ 运行状态不再只由 CPU、内存和 QPS 描述；
- ▶ 一次故障可能跨越模型、提示词、工具、服务和集群；
- ▶ 质量、延迟、成本与安全同时成为 SLO。

核心命题 模型版本、输入上下文和工具调用都会改变运行结果，需要与队列、缓存和硬件状态一起记录。

本讲学习目标

1. 说明 AI 原生系统的定义、组成和典型运行时特征；
2. 画出一个从用户请求到模型响应的端到端架构；
3. 区分训练、推理、RAG、Agent、记忆、多智能体与运维平台的职责边界；
4. 解释 GPU/异构算力为何成为系统设计的一部分；
5. 识别 AI 原生系统与传统微服务系统的关键差异；
6. 能用 ReAct、状态机和记忆等概念解释 Agent 的任务轨迹；
7. 为后续可观测性、故障检测和根因分析建立共同语言。

评价标准：不仅能复述组件名称，还能说明假设、边界、取舍和可观测证据。

一条请求的时间账：首 Token 为什么等了 1.7 秒？

假设各阶段串行执行，时间戳来自同一单调时钟；Prefill 包含首 Token 采样；TTFT 指首 Token 等待时间，TPOT 指后续每 Token 平均耗时。

阶段	开始/结束（秒）	耗时	需要保存的状态
检索	0.0 / 0.2	0.2 s	索引版本、文档 ID
排队	0.2 / 1.2	1.0 s	请求 ID、队列位置
Prefill	1.2 / 1.7	0.5 s	输入长度、模型版本
后续生成	1.7 / 3.7	2.0 s	共输出 101 个 Token

$$TTFT = 1.7\text{ s}, \quad TPOT = \frac{3.7 - 1.7}{101 - 1} = 20\text{ ms}$$

推导与判断 首 Token 前的等待主要来自队列；仅把后续生成加速一倍，TTFT 仍为 1.7 秒。真实系统应单独记录网络传输及首 Token 到达时刻。

算例使用假设数据。

本讲路线图：从概念到运维问题

1. 课程定位与共同语言；
2. AI 原生系统的定义、组成与生命周期；
3. 训练、推理、RAG、Agent 和 GPU 集群的架构连接；
4. 一次请求的运行时状态与关键指标；
5. 企业故障处理流程与相关研究案例；
6. 课堂讨论、总结与下一讲预告。



一、什么是AI原生系统

先把“AI 应用”“AI 系统”和“AI 原生”分开。

从 AI 功能到 AI 原生系统

AI 功能 传统业务系统中增加一个分类、推荐或生成接口。

AI 系统 模型、数据与服务共同构成可部署的软件系统。

AI 原生系统 系统的核心状态、资源调度和交互方式从一开始就围绕模型与智能体设计。



一个工作定义

AI 原生系统是以模型推理或智能决策为核心运行环节，
以数据、知识、工具与反馈为主要状态，
并针对动态算力、质量不确定性与持续学习进行系统设计的
软件系统。

核心计算 每次生成会逐步增加输出，并持续占用计算与缓存资源。

核心状态 除数据库外，还需管理上下文、缓存和长期记忆。

核心反馈 上线后持续评估质量、监控运行状态，并据此更新版本。

智算集群与 AI 系统全景

算力设施 / 集群管理 / 训练与推理 / 检索、智能体与业务应用



智算集群展开：节点怎样连接，数据走哪条网络？

管理节点

Kubernetes: API Server、
etcd、Scheduler

Slurm: slurmctld

提交任务、分配资源、管理状态

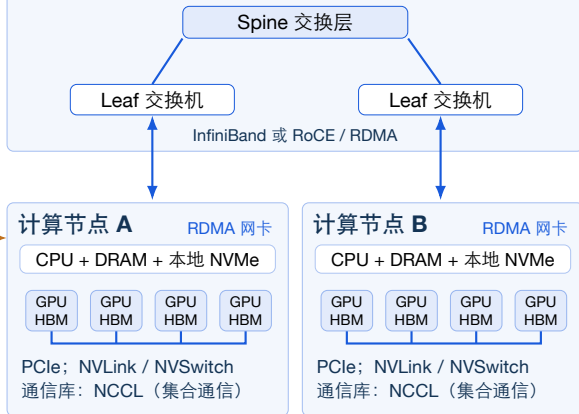
共享存储节点

Ceph: 对象存储
Lustre: 并行文件系统

训练数据、模型权重
检查点、实验产物

计算节点通过存储网络读写

计算网络：跨节点的梯度与中间结果



带外管理网络

BMC / Redfish

开关机、硬件状态、远程控制
操作系统无响应时，仍可经
独立管理通道诊断

机房与供电散热

机柜、PDU、UPS
风冷或液冷系统

检查功率、温度、供电告警

节点和交换设备都依赖这些
设施

存储网络：训练样本、权重与检查点

— 计算通信

— 存储读写

--- 控制与管理

同一套集群，训练与在线推理怎样使用资源？

环节	模型训练	在线推理
任务入口	提交训练作业、数据路径与参数	应用发来输入文本与生成参数
资源分配	Slurm 或 Kubernetes 分配作业资源	Kubernetes 等部署推理副本；服务端再调度请求
主要软件	PyTorch、DeepSpeed；按并行方案使用 NCCL	vLLM、SGLang；批处理与 KV 缓存管理
主要数据	读取训练样本；保存权重、优化器状态与进度	加载权重；保存请求上下文与 KV 缓存
跨卡通信	数据并行同步梯度；其他并行方案交换中间结果	模型跨卡时交换中间结果；独立副本可分别服务
运行指标	每步耗时、样本吞吐、损失、检查点保存时间	首 Token 延迟、生成间隔、达标吞吐、回答质量

同样的 **GPU** 利用率，对应不同问题 训练可能等最慢的通信进程；推理可能被请求排队或显存中的 KV 缓存限制。诊断要先识别工作负载。

沿着一次知识问答，读懂全景图

1. 上线前分配资源：Kubernetes 把推理 Pod 放到可用节点；容器通过驱动与计算库使用 GPU。调度器通常不参与每个 Token 的生成。
2. 收到问题后检索：应用通过检索组件查询文档或向量库，把选出的内容与问题组成模型输入；普通问答不必经过 Agent 编排。
3. 模型推理：vLLM 等服务安排请求与批处理，在 GPU 上计算并保存 KV 缓存，再返回生成结果；多 GPU 通信取决于具体并行方案。
4. 检查结果：用调用记录区分检索耗时、排队和生成耗时，并检查回答是否引用正确、是否完成用户任务。

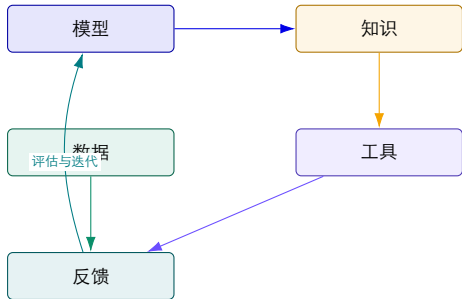
不同阶段，看不同指标 部署时看资源与模型加载；请求进入后看检索、排队、首 Token 和后续生成；返回后检查答案质量。

传统云原生与 AI 原生：重心发生了什么变化？

维度	传统云原生系统	AI 原生系统
主要工作负载	确定性业务逻辑、数据库事务	模型推理、检索、规划与工具调用
主要状态	请求、会话、数据库与缓存	上下文、KV Cache、向量、记忆、模型版本
资源瓶颈	CPU、内存、网络、磁盘	GPU 显存、带宽、批处理窗口、模型加载时间
质量定义	正确性、可用性、延迟	正确性、相关性、幻觉率、安全性、延迟与成本
故障表现	错误码、超时、实例宕机	输出质量下降、漂移、幻觉、工具误用与资源抖动

结论 系统评估既要看资源和性能，也要检查回答质量及运行时状态。

AI 原生系统的五类基本对象



模型
预测与生成

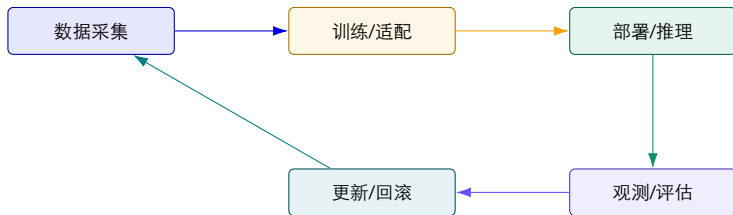
数据
训练与运行

知识
检索与记忆

工具
感知与行动

反馈
质量与运维

AI 原生系统的生命周期不是一条直线



运维含义 每一次线上请求都可能反过来影响数据、评估集、提示词、知识库和下一次模型发布。

二、从训练到推理：模型如何进入系统

训练产生能力，推理把能力变成服务，运维检查服务质量并处理运行故障。

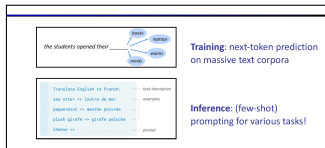
自回归语言模型：预测下一个 token

$$P_{\theta}(x_t | x_{<t})$$

给定已经看到的 token，模型输出下一个 token 的概率分布。

- ▶ 训练：在大规模语料上最小化 next-token prediction loss；
- ▶ 推理：把上一步输出追加到上下文，再重复一次预测；
- ▶ “会做题、会写代码”是规模、数据和训练目标共同产生的泛化能力，不是数据库查询。

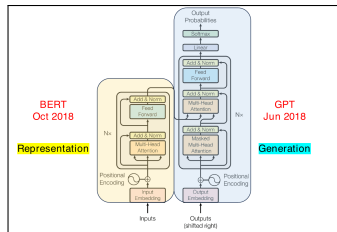
系统含义 每增加一个输入 token 都可能增加 Prefill 计算和 KV Cache；每增加一个输出 token 都会增加 Decode 时间、带宽与成本。



上：预训练学习语言分

布；下：推理时用 prompt 与示例完成 in-context learning。

Transformer: 把 token 序列变成可采样的概率分布



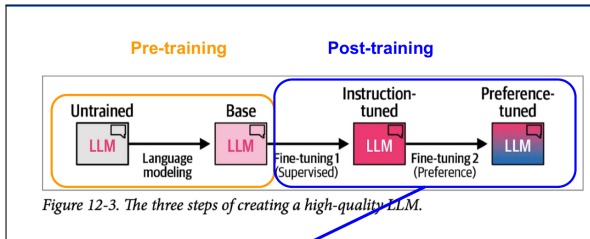
图中对比不同结构；本节生成过程

采用自回归解码。

1. Tokenizer 把文本映射为离散 token;
2. Embedding 与位置编码把 token 变成向量序列;
3. Self-attention 让每个位置读取上下文;
4. 线性层产生 logits, Softmax 将其转为下一 token 的概率。

不要混淆 模型结构决定“如何计算”；权重决定“学到了什么”；Serving 决定“如何在有限资源上执行”。

从 Base Model 到可用模型：能力、行为与成本逐步改变



独立截取“预训练—指令微调—偏好/推理对齐”流程图。

阶段	系统上新增的约束
预训练	数据、并行训练、检查点、收敛与数据血缘
SFT	指令格式、任务覆盖、数据质量与回归集
偏好/对齐	奖励定义、安全边界、拒答行为与稳定性
推理训练/蒸馏	训练数据、能力迁移；部署后再评估推理预算与成本

教学结论 “换一个模型”不是只换权重：输出分布、工具调用倾向、token 数和故障模式都会变化。

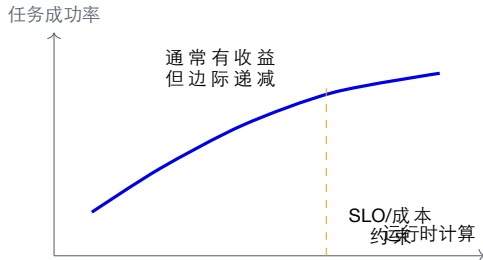
推理模型：把更多计算放到运行时

普通生成模型

- ▶ 一次或少量采样得到答案；
- ▶ 延迟主要由输入长度、输出长度和服务排队决定；
- ▶ 适合稳定、短链路和高并发请求。

推理/思考模型

- ▶ 允许更长的中间推理或自验证轨迹；
- ▶ test-time compute 换取复杂任务成功率；
- ▶ token、KV、工具等待和尾延迟都可能放大。



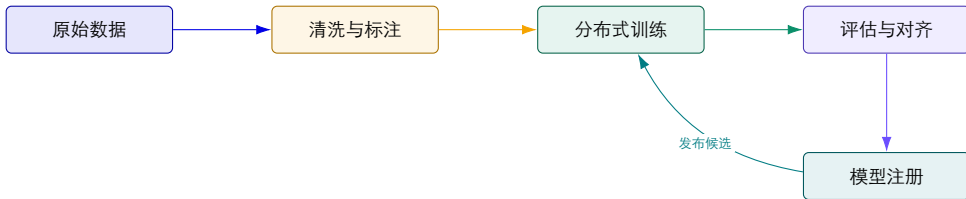
运维问题 模型路由器必须同时知道任务难度、预算、SLO 和当前 GPU/队列状态。

采样参数不是“玄学”：它们改变输出与容量

参数	作用	对质量/行为的影响	对系统/诊断的影响
Temperature	缩放 logits 分布	高值通常增加采样多样性；事实正确性需单独评测	同一输入更难逐 token 重放；评测需固定种子/配置
Top-p / Top-k	截断候选集合	控制采样尾部，影响创造性与格式稳定性	改变 token 分布和生成长度，导致成本与尾延迟变化
Max tokens	输出上限	可能截断答案或限制长推理	直接约束 Decode 时间、KV 占用与队列容量
Stop / tool schema	结束条件与结构约束	减少越界输出，错误 Schema 会阻断工具链	必须记录 finish reason、解析错误和重试路径

课堂例子 把 temperature 从 0.2 改到 0.9 后“偶尔答错”并不一定是模型回归；先检查请求配置、采样种子和路由版本。

训练阶段：模型能力如何被“生产”出来？

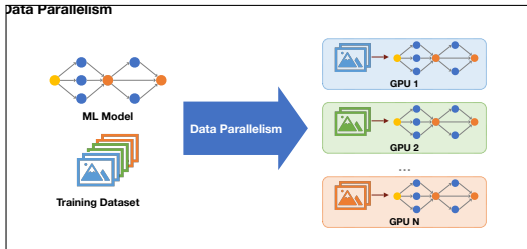


系统问题 数据规模、GPU 通信、检查点和失败恢复。

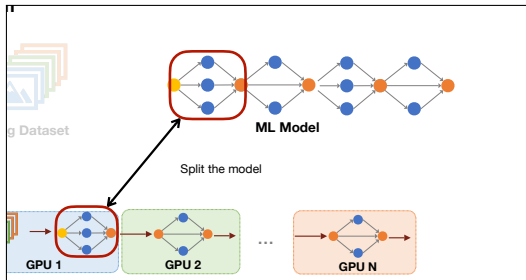
质量问题 训练集、评估集、偏差和安全边界。

运维问题 版本、血缘、资源成本与发布审计。

大模型训练为什么需要多种并行方式？



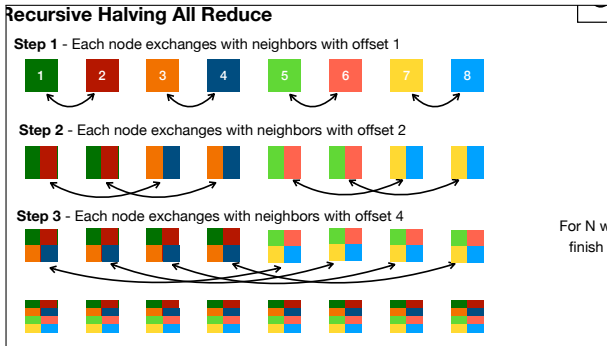
数据并行：复制模型，切分训练数据。



模型并行：切分模型，让单卡不必容纳完整参数。

系统取舍 数据并行受梯度同步开销约束；模型并行受跨卡通信、切分策略和负载均衡约束。实际大模型训练通常组合数据、张量、流水线与序列并行。

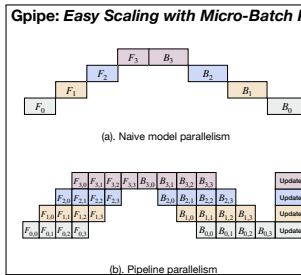
数据并行的一类瓶颈：梯度 All-Reduce



- ▶ 每个数据并行 rank 只计算本地 mini-batch 的梯度；
- ▶ All-Reduce 聚合所有梯度，并把结果重新分发给每个 rank；
- ▶ 递归减半等算法减少通信轮次，但性能仍取决于消息规模和网络拓扑；
- ▶ 慢节点、链路拥塞或一个 NCCL 超时都可能阻塞整个训练步。

需要观测 通信时间占比、计算通信重叠率、rank 间偏斜、有效带宽、NCCL 重试与链路错误。

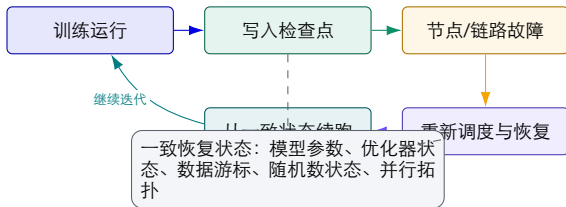
流水线并行：用微批提高利用率，也引入新的系统状态



- ▶ 把模型层切成多个 stage，放到不同设备；
- ▶ 把一个 batch 再切成 micro-batch，交错执行前向与反向；
- ▶ 微批越多，流水线气泡通常越小，但调度和激活内存压力上升；
- ▶ 最慢 stage 决定整体吞吐，分区不均会造成 GPU 空闲。

运维证据 除了 loss，还要观察各 stage 的计算时间、通信时间、气泡比例、激活内存和重计算开销。

大规模训练首先是一个长期运行的分布式作业



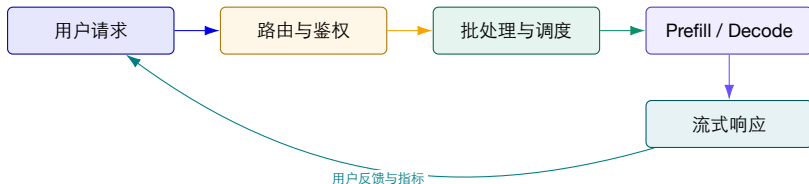
例：第 18,000 步发生 NCCL 超时。若只恢复模型参数而没有恢复数据游标，样本可能被重复或跳过。

可靠性目标 训练平台要把“数天或数周的计算”变成可续跑、可审计、可定位的分布式任务，而不只是启动一组 GPU 进程。

失败不只来自模型

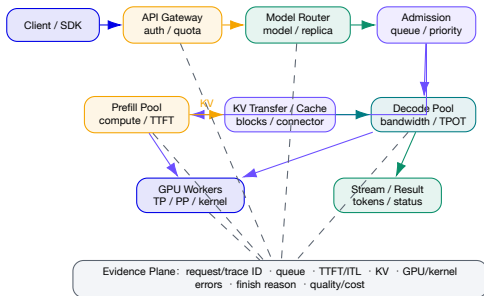
- ▶ 硬件故障、通信超时和系统升级都可能中断预训练；
- ▶ checkpoint 决定最多丢失多少计算；
- ▶ 自动恢复需要同时恢复模型、优化器、数据位置和随机状态；
- ▶ loss 正常也不代表通信、精度或数据管道没有隐藏错误。

推理阶段：模型能力如何变成在线服务？



推理服务的本质 把昂贵的模型计算封装成一个有延迟、并发、显存和质量约束的在线系统。

一页看懂生产级 LLM Serving：控制面、数据面与证据面



读图顺序

1. 网关先确定身份、配额与请求格式与限制；
2. Router/Admission 决定模型、实例、排队和缓存复用；
3. Prefill 处理输入，Decode 按步生成；
4. 解耦时 KV 成为跨池数据；
5. 每层必须以同一 request/trace ID 回到证据面。

两类箭头 实线是请求/KV/Token 数据流；虚线是观测证据流。

训练服务与推理服务：同一个模型，两种系统

维度	训练系统	推理系统
目标	最小化损失、提升能力	满足质量、延迟、吞吐和成本目标
工作负载	长时间、大规模、批量计算	短请求、高并发、动态长度
关键资源	GPU 数量、通信带宽、检查点存储	GPU 显存、KV Cache、调度和网络
故障恢复	检查点、重启、任务重试	超时、限流、降级、回滚与多副本
主要证据	loss、吞吐、收敛、评估集	TTFT、TPOT、P99、错误率、质量与成本

运维启示 不能用训练系统的指标去替代推理系统的 SLO；二者共享模型，却不共享同一套运行假设。

推理请求的两个阶段：Prefill 与 Decode

Prefill：读入上下文

- ▶ 处理用户输入、历史对话和检索结果；
- ▶ 计算首个 token 前的大量矩阵运算；
- ▶ 主要影响首 token 延迟（TTFT）。

Decode：逐 token 生成

- ▶ 每一步读取并更新 KV Cache；
- ▶ 受显存容量、带宽和调度影响；
- ▶ 主要影响输出速度与尾延迟。



连续批处理：推理调度从“整批等待”变成“逐轮补位”

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4					

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

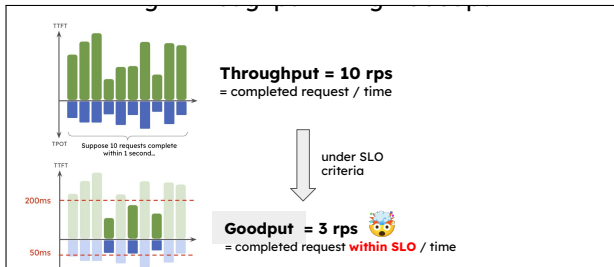
T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	END	
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5

- ▶ 静态批处理必须等待批内最长请求完成；
- ▶ 连续批处理在每次生成迭代结束后，移出完成请求并接纳新请求；
- ▶ GPU 利用率提高，但调度器必须管理不同长度、优先级与显存状态；
- ▶ 高负载下仍要限制排队，避免吞吐优化转化为尾延迟失控。

需要观测 waiting/running 请求数、batch token 数、抢占与换出次数、TTFT、TPOT 和 P99。

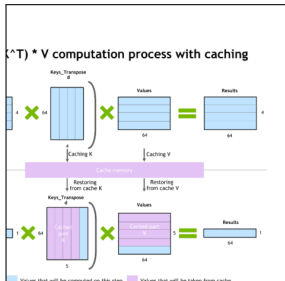
高吞吐不等于好服务：用 Goodput 约束推理容量



- ▶ Throughput 统计单位时间完成的全部请求；
- ▶ Goodput 只统计满足 TTFT、TPOT 等 SLO 的完成请求；
- ▶ 提高到达率可能增加“完成量”，却让请求超出用户可接受的延迟；
- ▶ 容量规划应寻找满足 SLO 的最大负载，而不是压出最高 QPS。

运维含义 只汇报吞吐会掩盖排队和尾延迟；服务是否健康必须同时看吞吐、SLO 达标率和拒绝/超时。

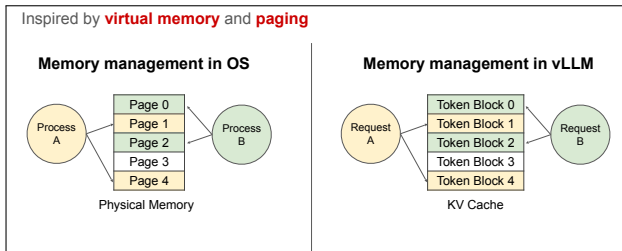
为什么要单独管理 KV Cache?



- ▶ Prefill 为输入 token 生成各层 Key/Value;
- ▶ Decode 每一步读取历史 KV, 只计算新增 token;
- ▶ 缓存规模随层数、上下文长度、并发和精度增长;
- ▶ 长短请求混合会带来碎片、预留浪费和抢占。

工程含义 推理优化不仅是“让模型跑得更快”，还包括显存管理、缓存复用和请求调度。

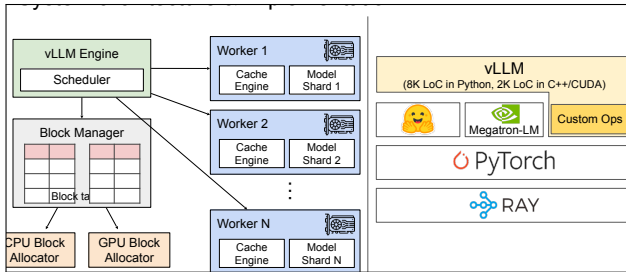
PagedAttention: 像操作系统分页一样管理 KV Cache



- ▶ 将一个请求的 KV Cache 切成固定大小 token block;
- ▶ 逻辑上连续的 token, 可以映射到不连续的物理显存块;
- ▶ block table 负责地址映射, 减少外部碎片和大块预留;
- ▶ 共享前缀时可复用物理块, 并通过引用计数和写时复制隔离。

类比的边界 PagedAttention 借鉴虚拟内存思想, 但服务调度仍要处理 GPU kernel、批次和模型执行的时序约束。

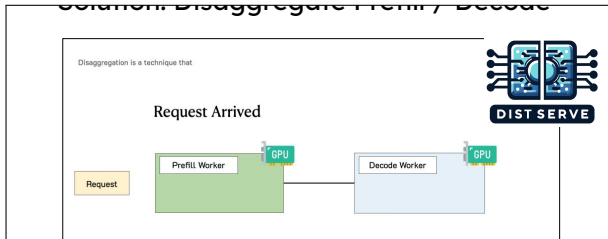
推理引擎不是一个模型进程：调度、缓存与执行相互耦合



- ▶ Scheduler 决定哪些请求进入下一轮；
- ▶ Block Manager 管理 CPU/GPU 缓存块及映射；
- ▶ 多个 Worker 持有模型分片和本地 Cache Engine；
- ▶ 张量并行通信、缓存分配和请求队列共同决定尾延迟。

观测对象 请求队列、调度决策、block 分配、swap/preemption、worker 健康和跨卡通信都应进入同一证据链。

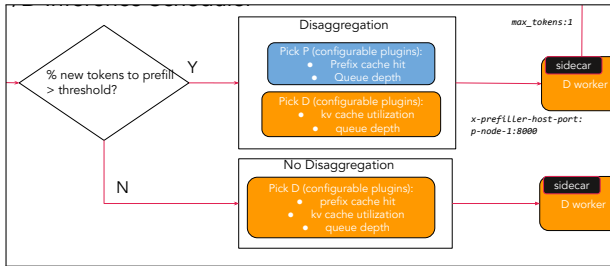
Prefill/Decode 解耦：用不同资源池服务不同阶段



- ▶ Prefill 偏计算密集，Decode 偏显存带宽和批处理；
- ▶ 解耦后可分别选择 GPU 数量、并行策略和批处理策略；
- ▶ 新请求先进入 Prefill Worker，再把 KV 状态交给 Decode Worker；
- ▶ 收益来自减少阶段干扰，代价是新增路由、传输和跨池故障。

适用边界 P/D 解耦不是所有负载的默认答案；上下文长度、输出长度、网络 and SLO 共同决定是否值得。

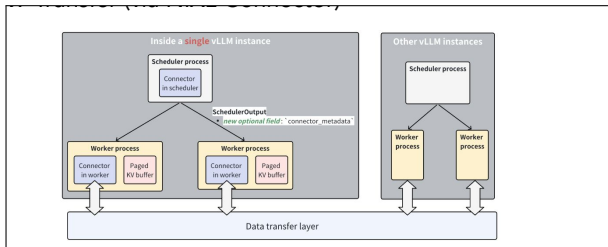
llm-d: 按请求状态选择推理副本



- ▶ P/D-aware Scheduler 判断请求是否需要独立 Prefill;
- ▶ 选择 Prefill Worker 时可参考前缀命中和队列深度;
- ▶ 选择 Decode Worker 时可参考 KV Cache 利用率与负载;
- ▶ Routing Sidecar 将 Prefill 结果和目标信息交给 Decode 节点。

平台变化 Kubernetes 调度器放置 Pod; llm-d 的请求路由根据模型、队列和 KV 状态选择服务副本。两者的调度对象不同。

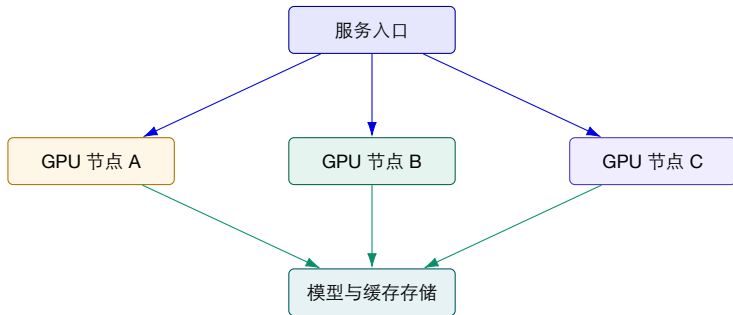
KV 跨节点传输：解耦架构新增了一条关键数据路径



- ▶ Prefill Worker 产生的 KV 数据必须交给选定的 Decode Worker;
- ▶ NIXL 等连接器抽象 GPU、CPU 与网络间的数据传输;
- ▶ 放置策略决定数据走 IPC、NVLink、RDMA 还是跨机网络;
- ▶ 传输阻塞会直接推高 TTFT, 并可能让 Decode GPU 空等。

新增证据 KV 字节数、传输时延、链路类型、连接器状态、目标 Worker、超时和回退路径必须随请求记录。

GPU 集群：计算资源本身就是系统状态



容量 显存决定模型和上下文能否放下。

带宽 GPU、CPU、网络和存储之间的搬运决定延迟。

调度 多租户请求需要公平、隔离和可回收。

从单机 GPU 到异构算力集群

资源异构

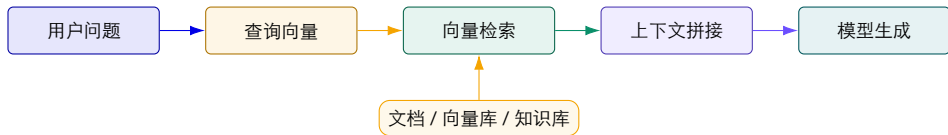
- ▶ 不同 GPU 型号、显存、互联和驱动；
- ▶ CPU、GPU、NPU 与专用推理加速器并存；
- ▶ 资源价格、能耗与性能不同。

系统异构

- ▶ 模型大小、量化方式和上下文长度不同；
- ▶ 工作流可能调用多个模型和外部工具；
- ▶ 任务有实时、批量和离线三种节奏。

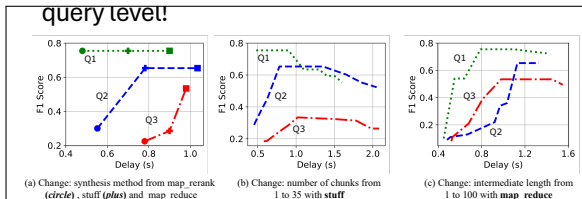
资源画像 + 工作负载画像 + 调度策略 = 可预测的服务质量

RAG：把外部知识接入模型运行时



运维视角 RAG 的质量不仅由模型决定，还由文档新鲜度、切分、召回、权限和上下文预算决定。

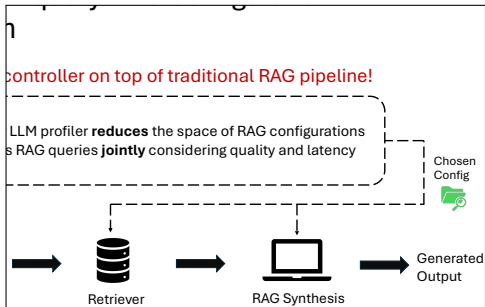
RAG 是质量、延迟与资源之间的在线权衡



- ▶ 检索 chunk 数量、综合方式和中间文本长度都会改变答案；
- ▶ 同一个配置对不同查询的收益不同；
- ▶ 增加检索与重排可能提高质量，也会增加 token、GPU 时间和尾延迟；
- ▶ 生产系统需要同时记录查询、配置、召回证据与最终质量。

系统结论 RAG 不是“向量库 + LLM”的固定流水线，而是需要按请求治理质量—成本—延迟的组合系统。

RAG 控制器：配置选择本身也成为在线决策



- ▶ Profiler 根据查询特征缩小可选配置空间；
- ▶ Scheduler 联合考虑答案质量、延迟和当前资源；
- ▶ 控制器为每个请求选择 chunk 数量、综合方法等配置；
- ▶ 配置错误可能不报系统异常，却持续产生低质量或高成本答案。

可诊断性 要复现一次 RAG 失败，必须知道控制器输入、候选配置、最终选择、资源状态和召回文档。

三、从大模型到智能体：完成任务的基本方式

模型负责生成，运行时负责状态，工具和环境提供反馈。

LLM、工作流和 Agent：三个概念不要混为一谈

对象	谁决定下一步？	适合什么任务？	主要故障面
LLM 调用	程序给定 prompt 与停止条件	单轮生成、分类、摘要、结构化抽取	幻觉、格式错误、延迟与 token 成本
工作流	程序/状态机预先定义分支	流程稳定、审批明确、需要可审计	分支覆盖不足、状态遗漏、异常路径僵化
Agent	模型依据状态选择工具/下一步	开放式、多步骤、环境反馈强的任务	计划漂移、工具误用、循环、错误完成



边界判断 如果下一步完全由代码决定，它更像工作流；如果模型可以选择行动，但没有外部状态与验证，它仍不是可靠的生产 Agent。

ReAct: 把“想”和“做”交替起来



Reasoning 更新内部信念；Acting 从环境获得外部反馈；ReAct 交替执行二者。

1. Thought: 当前证据说明了什么？
2. Action: 选择一个工具与参数；
3. Observation: 工具返回真实结果或错误；
4. 循环: 把结果加入上下文，直到满足完成条件。

为什么有效 工具返回值提供了“接地”反馈，减少模型只凭参数记忆回答的风险。

为什么危险 每一次循环都会消耗 token、KV 和外部资源；错误观察可能让后续计划越走越偏。

一个可执行的 Agent 轨迹：查询数据库并验证结果

任务：列出最高年级全部学生的姓名和年级。

Thought 1：先查看可用表；

Action 1：execute(SHOW TABLES);

Observation 1：发现表名为 highschoolers;

Thought 2：查看字段，避免猜列名；

Action 2：execute(DESC highschoolers);

Observation 2：字段含 name, grade, 其中 grade 表示年级；

Thought 3：查询最高年级并检查输出；

Action 3：SELECT name, grade FROM highschoolers
WHERE grade=(SELECT MAX(grade) FROM highschoolers);

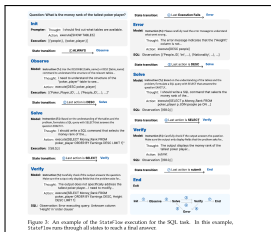
Verify：检查最高年级条件及全部并列学生，再提交。

轨迹中需要记录什么？

- ▶ 每次模型调用的模型版本、输入摘要、token 和耗时；
- ▶ 工具名称、Schema 版本、参数摘要、返回码和错误；
- ▶ 状态转移原因与停止条件；
- ▶ 最终结果是否由环境状态或业务 oracle 验证。

教学重点 “SQL 执行成功”
是中间事件，不是任务成功；
Verify/Submit 才是端到端完
成边界。

固定流程还是状态机？StateFlow 的启发



状态：Init → Observe → Solve →

Verify → End；错误时转入 Error，再回到 Solve。

- ▶ 用显式状态承载当前阶段，避免每轮都把所有历史交给模型；
- ▶ 用规则或模型决定状态转移；
- ▶ 为错误、验证和终止单独建状态，形成可审计边界；
- ▶ 状态机不是“限制智能”，而是把可控部分显式化。

结果如何读 StateFlow 论文在 InterCode 上报告：相比 Plan&Solve、ReAct 等基线，状态化流程提高成功率，并在示例中减少交互成本；具体数字依模型和数据集而变。

Agent 记忆：上下文窗口不是长期记忆

- Even with massive 1M+ token windows, fundamental problems remain.
 - Token cost and TTFT
 - "Lost in the Middle" (Liu et al., 2023): Information at the **beginning** and **end** of the prompt is well used, but not in the middle.
 - Prompting is stateless unless externally managed
 - Two Types of "Context" / "Memory":
 - **1. In-Context Memory (Short-Term):** The information fed directly into the prompt.

长上下文仍有 token 成本、首 token 延迟和“中间信息不易被利用”等问题。

记忆类型	例子	典型读写策略
工作记忆	当前任务、工具结果、最近对话	随请求写入上下文；受窗口与 KV 预算限制
情景记忆	过去一次排障的轨迹与结果	按相似任务检索；需要时间、租户和权限过滤
语义记忆	稳定事实、文档、服务拓扑	Embedding/关键词/图检索 + 版本管理
程序记忆	Skill、脚本、提示模板、策略	版本化、测试、审批后才能执行

关键问题 什么值得写入？何时过期？谁能读/改/删？记忆写错会不会成为下一次行动的“事实”？

从被动 RAG 到主动记忆：Agent 需要管理记忆

被动检索 (RAG)

- ▶ 系统依据当前 query 自动取 top-k;
- ▶ 读路径稳定、容易接入;
- ▶ 不知道哪些内容已经在上下文中;
- ▶ 通常不会自主修改、连接或遗忘记忆。

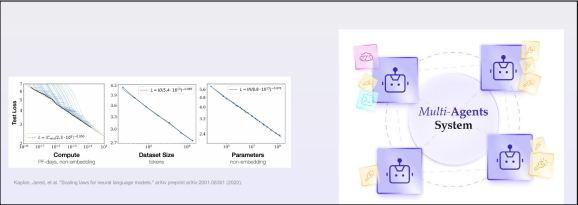
主动记忆 (Agent memory API)

- ▶ Agent 选择记住、检索、更新或忘记什么;
- ▶ 可以把经验抽象成事实、策略或 Skill;
- ▶ 需要写入门槛、冲突解决、TTL、审计和回滚;
- ▶ 记忆管理本身会产生额外模型调用和故障面。



课堂例子 排障 Agent 不能把“一次偶然的 403”直接写成永久事实；应记录证据、时间、影响范围和验证结果，再决定是否保存为后续参考。

多智能体不是“多开几个聊天窗口”



研究问题：规模、角色分工、通信拓扑是否带来可重复的能力收益？

拓扑	适用与代价
中心编排	Planner/Manager 分派任务；易治理，但中心成为瓶颈
层次协作	领域 Agent 向上汇报；适合复杂组织，传播延迟更高
Peer-to-peer	Agent 直接协商；可减少中心依赖，但通信与冲突难控
黑板/共享状态	通过共享记忆异步协作；便于解耦，需解决一致性与污染

工程判断 引入多智能体前，应检查任务能否分解、角色是否互补、结果如何合并，并实测收益是否超过通信与协调成本。

斯坦福小镇：25 个 Agent 在同一环境中生活



先读图

地图是共同环境；放大框展示不同角色的观察和行为。

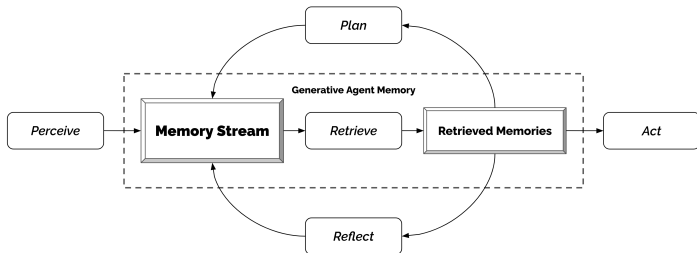
再看实现

每个角色有自己的经历记录，并据此制定计划、与其他角色交流。

研究范围

这是交互式社会行为模拟，不能直接证明生产运维中的正确率或可靠性。

从个人记忆到下一步行动



观察与记忆 把经历写入记录；检索结合相关性、时间和重要性。

计划与反思 计划安排后续行动；反思归纳已有经历，结果也进入记忆。

运维中的检查点 检索了哪条记录？记录是否过期？行动依据能否追溯？这是课程延伸问题。

聚会实例：信息怎样在多个 Agent 之间传播？



论文观察

从一个角色想办聚会的初始设定出发，邀请经对话传播，部分角色随后到场。

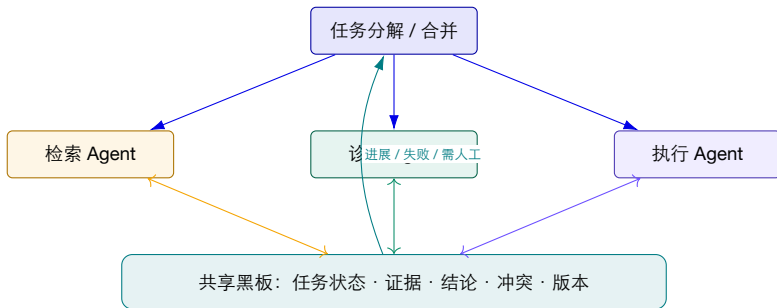
课堂追问

若把“邀请”换成“故障通报”，应记录谁收到消息、采用哪版事实、是否完成分配的任务。

区分两个结论

出现协调行为，不等于消息必达、结论一致或任务必定成功。

多智能体协作的新增状态：消息、角色与共享事实



消息 谁发给谁、何时发、是否丢失或重复？

一致性 不同 Agent 看到的拓扑、版本和故障状态是否一致？

合并 多个局部结论如何形成可验证的全局动作？

Agent 评测：不能只看“最后答对了吗”

Success rate (higher is better)		Agent Curriculum (full benchmark)				
Task Category (task count)		GPT-3.5	GPT-4o	GPT-4o-v	Llama3	Mixtral
WorkArena L3 (235)						93.9 ±3.4
Contextual Understanding (32)						87.5 ±11.7
Data-driven Decision-Making (55)						100.0 ±0.0
Planning and Problem Solving (44)						87.5 ±11.7
Information Retrieval (56)						100.0 ±0.0
Sophisticated Memorization (48)						91.7 ±8.0
WorkArena L2 (235)						93.9 ±3.4
Contextual Understanding (32)						100.0 ±0.0

WorkArena++ 将企业任务拆成上下文理解、决策、规划、检索和复杂记忆等类别；结果显示与人类仍有明显差距。

层次	示例指标
结果	任务成功率、业务状态、答案正确性
轨迹	工具成功率、无效循环、重试、步数、验证覆盖
资源	token、GPU 时间、外部 API、人工接管
安全	越权调用、敏感数据、危险动作拦截
可复现	种子、模型/工具/知识库版本、环境快照

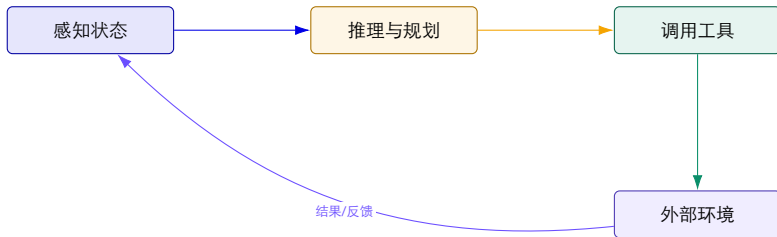
评测原则 如果只看最终答案，会把“碰巧成功”和“可复用、可解释、可安全执行”混在一起。

Agent 常见失败：错误回答、重复调用与无法停止

失败模式	典型表现	首要证据/保护措施
计划漂移 工具误用 记忆污染 循环/不终止 错误完成 多 Agent 冲突	任务越做越偏，目标被中间结果替代 参数 Schema 合法但语义错误，或反复重试 错误事实被长期复用 重复同一搜索或调用，成本持续上升 产生回答但外部状态未改变或未验证 角色结论互相矛盾，共享黑板过期	原始目标、子目标版本、状态转移与预算 工具版本、参数摘要、返回值、重试退避 写入来源、置信度、TTL、冲突和删除记录 去重键、最大步数、token/时间预算、停止条件 业务 oracle、变更前后状态、人工确认 消息顺序、状态版本、冲突解决与仲裁者

桥接后续章节 后续 AgenticSRE 将把这些失败转成可注入、可观测、可验证的实验；本讲先建立概念和证据语言。

Agent 如何根据工具结果决定下一步?

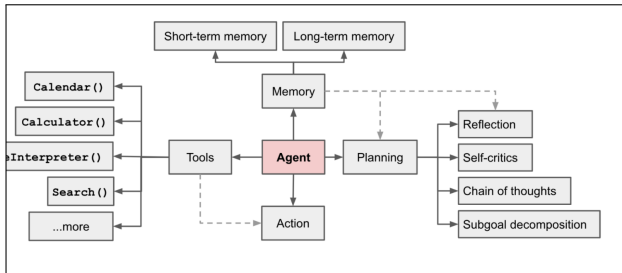


新状态 工具返回值会改变下一步上下文。

新风险 工具权限、提示词注入与错误行动。

新指标 轨迹长度、工具成功率、人工接管率。

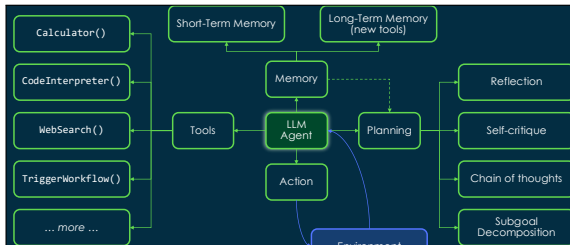
Agent 系统：模型只是规划、记忆、工具与行动的中枢



- ▶ **Planning** 把目标分解为可执行步骤；
- ▶ **Memory** 保存短期上下文、历史经验和长期知识；
- ▶ **Tools** 把模型能力扩展到搜索、代码、数据库和业务 API；
- ▶ **Reflection** 根据结果修正计划，形成多轮状态循环。

运维变化 一次任务包含多次模型调用和工具调用；部分步骤成功并不代表端到端任务正确，必须保存完整轨迹。

生产级 Agent：谁维护状态，谁检查权限？

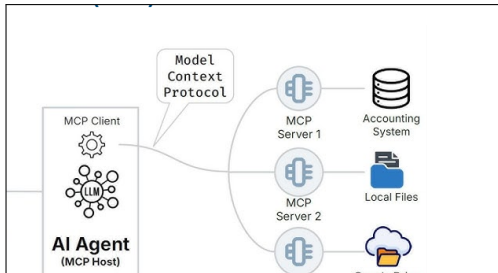


独立裁取组件关系：Tools、Memory、Planning、Action 与 Environment。

1. **Task/Intent**：把自然语言目标变成范围、约束和终态；
2. **Agent Runtime**：保存计划、状态、预算和停止条件；
3. **Model Gateway**：负责模型路由、重试、成本和版本；
4. **Memory/RAG**：提供工作记忆、历史轨迹和带来源知识；
5. **Tools/Environment**：查询或改变外部世界；
6. **Policy/HITL**：在执行前检查权限、风险、审批和回滚；
7. **Trace/Eval**：记录决策—工具—证据—结果，用于重放和评测。

关键边界 模型提出下一步；Runtime 维护状态；
Policy 决定能否执行；Outcome/Oracle 判断是否真的完成。

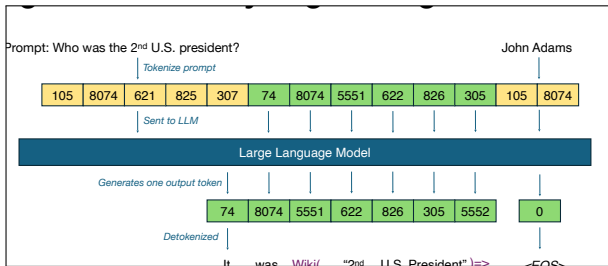
MCP: 降低工具接入复杂度，也扩大运行时故障面



- ▶ MCP Host 中的 Agent 通过 Client 连接多个 Server;
- ▶ Server 将数据库、文件、云服务和企业系统封装成工具与资源;
- ▶ 标准接口减少重复适配，却不能自动保证权限、正确性和可用性;
- ▶ 任一外部调用的延迟、返回值或权限变化都可能改变后续轨迹。

需要新增的证据 工具版本、参数与返回摘要、耗时、授权、重试和人工确认必须与模型请求关联。

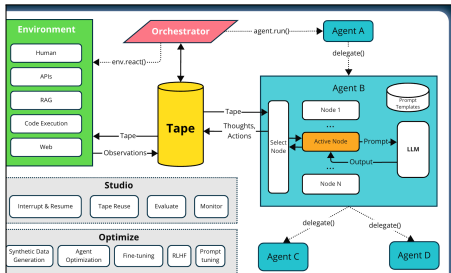
工具调用会打断模型生成：Agent 推理是交替执行的工作流



- ▶ 模型生成工具调用后，token 生成暂停；
- ▶ 外部系统执行搜索、计算或业务动作；
- ▶ 工具结果写回上下文，模型再继续生成；
- ▶ 等待期间 KV Cache 可以保留、换出或丢弃后重算，各有不同成本。

系统结论 Agent 请求不是一段连续 GPU 计算，而是模型阶段与非模型阶段交替、可暂停和可恢复的状态机。

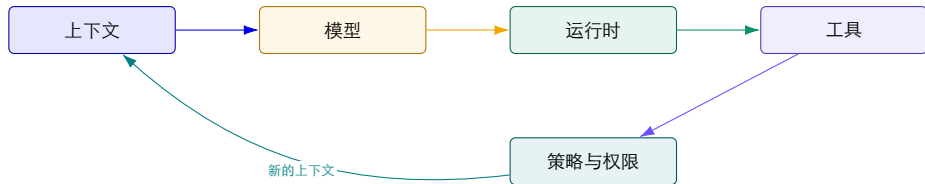
语义轨迹：Agent 的日志不应只是字符串拼接



- ▶ Tape 将 thought、action、observation 记录成结构化步骤；
- ▶ 环境和工具把执行结果继续写入轨迹；
- ▶ 多个 Agent、评估器和优化器可以消费同一条 tape；
- ▶ 结构化轨迹支持重放、归因、评估和训练数据生成。

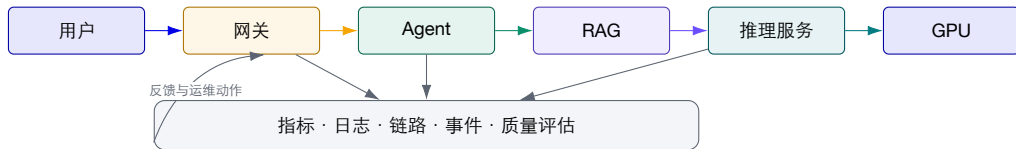
治理要求 轨迹可能包含提示词、工具参数和业务数据，必须同时考虑可观测性、脱敏、权限、保留周期和审计。

AI 原生系统的“运行时”到底包含什么？



关键转变 “运行时”不再只是进程和容器，而是上下文、模型、工具、策略和反馈共同组成的动态状态机。

示例：Agent 检索资料后调用模型的请求路径



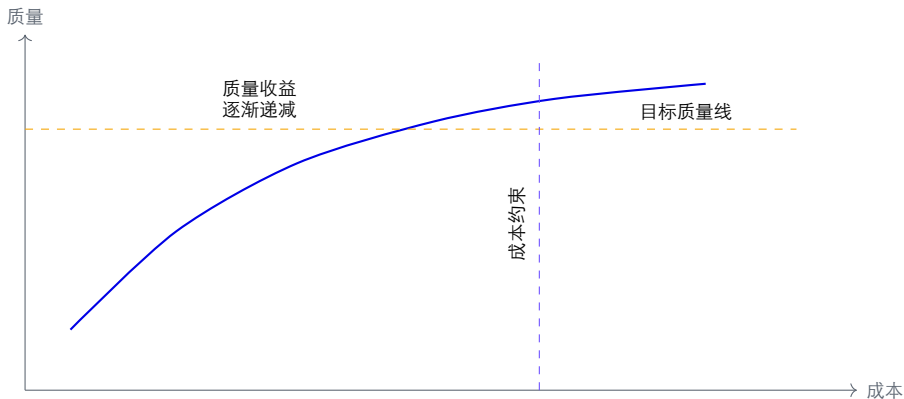
课堂练习：请在这条链路上标出三个最可能发生“隐性故障”的位置。

请求路径上的状态与边界

边界	进入时带来的状态	离开时需要留下的证据
网关 Agent RAG 推理服务 GPU 集群	租户、身份、限流、路由 历史上下文、任务目标、工具权限 查询、知识库版本、过滤条件 模型版本、批次、KV Cache 节点、显存、互联、驱动	请求 ID、策略决策、排队时间 轨迹、计划、工具调用与人工接管 召回文档、相关性、权限与上下文长度 TTFT、TPOT、token 数、错误码 利用率、显存水位、温度、拓扑与调度

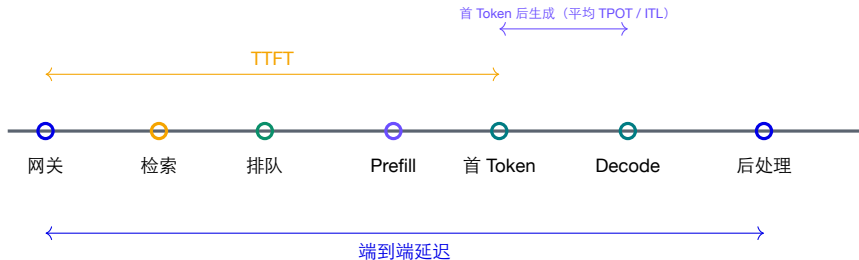
故障定位原则 没有边界证据，就没有可复现的诊断；没有版本证据，就无法解释质量变化。

质量不是一个数：AI 系统的多目标 SLO



系统取舍 提高模型规模可能改善质量，却也可能增加延迟、显存、能耗和故障半径。

延迟：用户感知的是整条链路



不能只盯着模型服务的平均耗时；排队、检索、工具调用和尾延迟共同决定用户体验。

成本：每个 token 都在消耗系统资源

直接成本

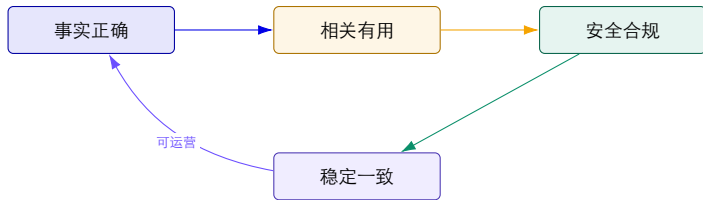
- ▶ GPU/加速器租赁与折旧；
- ▶ 电力、制冷、机房和网络；
- ▶ 模型调用、存储和数据传输。

隐性成本

- ▶ 空闲显存、低效批处理和重复检索；
- ▶ 人工审核、故障处置和回滚；
- ▶ 错误答案带来的业务与安全损失。

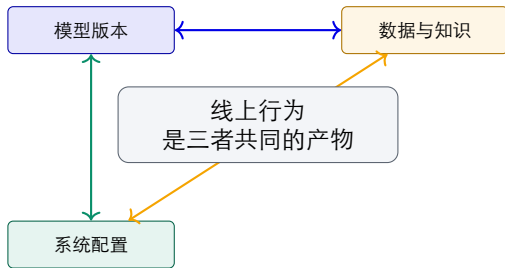
$$\text{Token} \times \text{GPU 时间} \times \text{并发与利用率} \rightarrow \text{单位请求成本}$$

质量：正确输出不等于高质量输出



运维含义 质量指标必须与模型版本、知识库版本、提示词、用户任务和工具轨迹关联起来。

模型、数据和系统之间存在强耦合

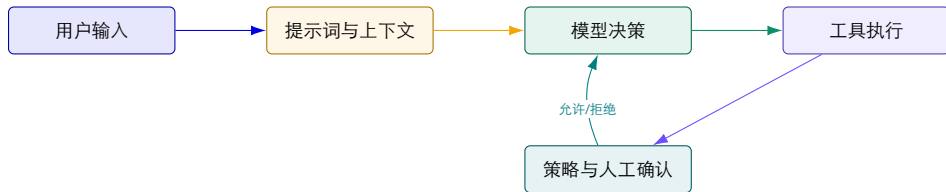


换模型 可能改变延迟、成本和回答风格。

换数据 可能改变召回、偏差和合规边界。

换配置 可能改变路由、权限和故障半径。

安全边界：Prompt、工具和权限共同决定风险



输入风险 Prompt 注入、越权指令、敏感数据。

决策风险 幻觉、错误规划、不可解释。

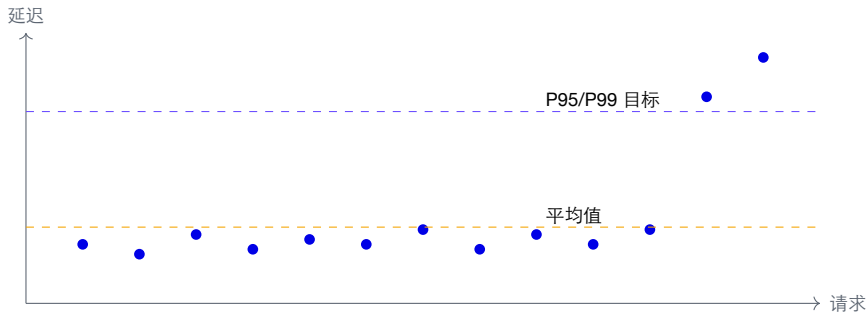
执行风险 危险工具调用、误操作、数据外泄。

故障也开始“跨层传播”



思考 如果只看“模型服务错误率”，能否解释这次故障？还需要哪些跨层证据？

一个简单但危险的误区：平均值掩盖了长尾



AI 服务的长尾可能来自检索、工具、批处理、显存回收或重试，而不只是模型本身。

系统边界：哪些事情必须由平台负责？

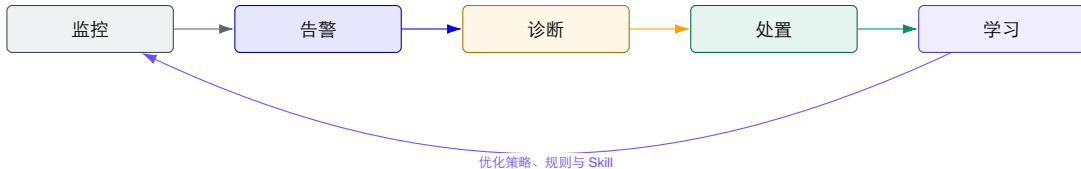
能力	平台应提供	业务/模型团队应负责
资源与调度	GPU 池、队列、配额、隔离、扩缩容	选择模型规格与资源需求
可观测性	统一指标、日志、链路、审计和查询	定义质量指标与业务阈值
发布与回滚	版本、灰度、回滚、变更记录	模型、提示词、知识库的发布内容
安全与治理	身份、权限、策略、人工确认	业务风险分级与合规规则
故障处置	告警、编排、自动化动作与证据链	领域判断、验收和复盘

平台化的价值 把重复任务写成标准流程，明确跨团队分工，并在执行危险操作前检查权限。

四、企业级智能运维的系统视角

把“看见问题”推进到“理解问题、协同处置、记录处理经验”。

监控发现问题后，怎样诊断、处置并检查恢复？

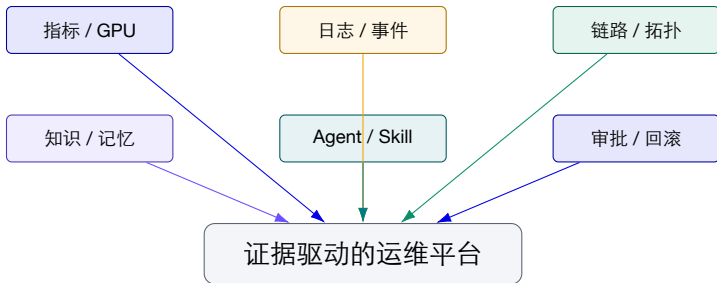


DevOps 缩短开发与运维反馈环。

SRE / AIOps 用 SLO 和多源数据治理可靠性。

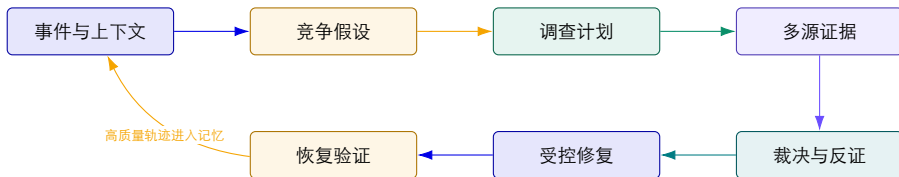
Agentic SRE 在证据约束下参与调查、执行和验证。

企业级平台：怎样用监控数据和工具处理故障？



平台化的价值 模型只负责部分判断；平台还必须负责数据接入、对象隔离、工具接口约定、权限审批、执行记录和恢复验证。

先提出可能原因，再查证据，最后检查恢复结果



假设 可证伪解释 + 预期
证据 + 调查计划。

证据 必须绑定对象、时
间和独立来源。

恢复检查 命令执行成功
不等于系统已经恢复。

智算案例：KV Cache 压力为何不能被简化为“GPU 忙”？

阶段	关键信息
上下文 竞争假设 关键证据 诊断链 最小恢复	长上下文负载启动后，KV Cache 与显存同步上升；无磁盘压力事件 H1 KV Cache 耗尽；H2 GPU 计算饱和；H3 请求队列过载；H4 模型进程异常 KV Cache 99%、GPU 显存 98%、waiting=12、TTFT P95=8.2s、TPOT=0.8s 长上下文占用 KV Cache → 新请求等待 → TTFT / E2E 退化 优先停止额外负载，验证 Cache、队列和 TTFT 回落；不无条件重启健康 vLLM

解释 显存高描述的是“容量压力”，GPU 利用率高描述的是“计算忙碌”；两者可能同时出现，也可能完全不同。

人的位置：诊断置信度不等于自动执行权限

诊断门

- ▶ 是否有多个独立证据源？
- ▶ 是否存在未解决的反证或冲突？
- ▶ 根因是否落到明确对象与时间窗？

执行门

- ▶ 动作风险、影响面和权限是否明确？
- ▶ 是否需要人工审批？是否可回滚？
- ▶ 执行后如何验证 SLO 和业务恢复？

安全原则 自愈目标不是“自动执行更多命令”，而是在证据充分时执行最小影响动作，并能证明系统已经恢复。

平台的三个核心抽象：对象、事件、流程

对象 (Object) 主机、服务、接口、Pod、模型、GPU、知识库。知识与证据必须归属到对象。

事件 (Event) 告警、变更、发布、异常和人工处置。事件记录对象状态如何变化。

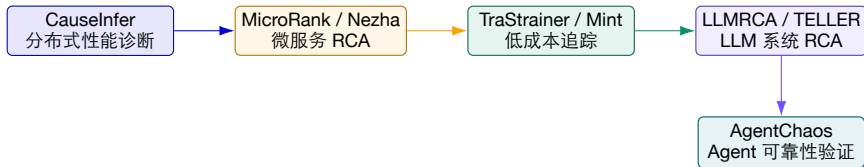
流程 (Workflow) 巡检、诊断、审批、执行、回滚与复盘。流程定义安全行动边界。

对象 × 事件 × 流程 = 可运营、可审计的智能

五、研究主线：从云原生诊断到 LLM 与 Agent 可靠性

用代表性论文理解课程模块为何这样连接。

代表性研究演进：系统变了，诊断对象也在变

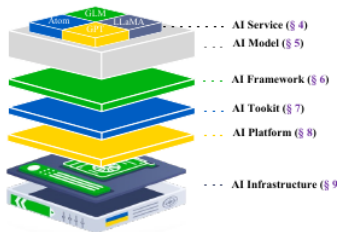


不变的问题 如何把端到端症状定位到可行动的根本因？

变化的系统 单体/分布式服务 → 微服务 → LLM 推理 → Agent。

增加的证据 指标与日志 → 链路与拓扑 → GPU 事件 → Agent 轨迹。

论文视角：AI 系统可以分成六个相互依赖的层



1. **AI Service**: 对用户交付的 AI 能力;
2. **AI Model**: 产生预测、生成和决策;
3. **AI Framework**: 定义并执行模型计算;
4. **AI Toolkit**: 连接框架和硬件, 如 CUDA;
5. **AI Platform**: 训练、部署和运营平台;
6. **AI Infrastructure**: GPU/TPU、网络和存储。

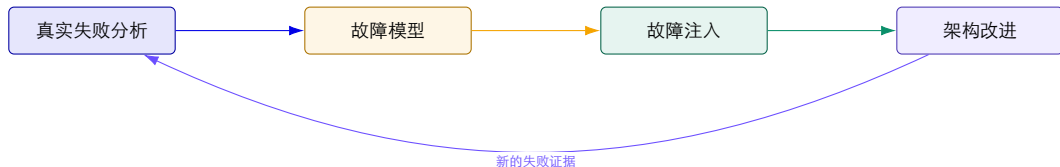
如何使用这个视角? 前面的七层图按部署组件组织; 这里按故障职责分六类, 两者不是同一套层号。它把“模型失败”拆成六层问题: 同一用户症状可能来自服务逻辑、模型、框架、工具链、平台或硬件。

这篇综述给运维带来的三个关键认识

先分析真实失败 论文系统分析了 142 项研究，按六层归纳生产 AI 系统中的失败。

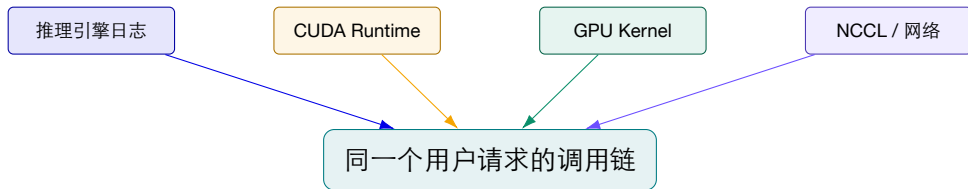
再检查测试能力 现有故障注入工具并不能覆盖所有已观察到的真实故障。

最后考虑跨层传播 数据预处理故障可能传播到训练、模型和服务，单层注入无法复现完整链路。



教学解释：可靠性不是“等故障发生再修”，而是把生产证据转化成可重复的验证实验。

TELLER 的问题意识：单层证据只能看到故障的一个切面

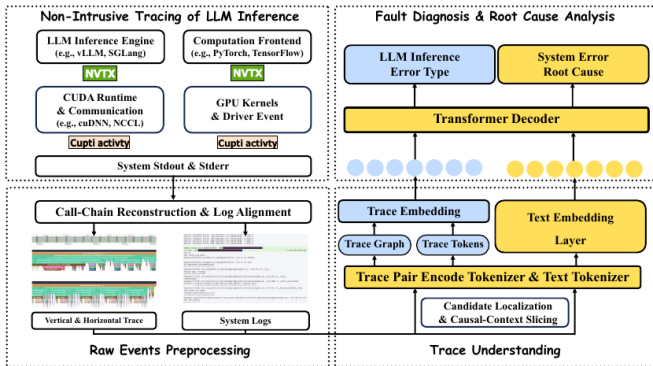


Profiler 看见 底层 kernel 与时间线，却缺少请求语义。

日志看见 调度和错误文本，却缺少设备层执行证据。

RCA 需要 按请求对齐跨层 trace 与 log，并保留因果结构。

TELLER: 非侵入采集、请求级重建、压缩后诊断



1. 用 NVTX/CUPTI 和日志采集引擎、框架、CUDA 与 GPU 事件，不修改模型二进制；
2. 将异构事件重建为请求级调用链，并把日志按请求和时间对齐；
3. 先定位可疑区域并切取因果上下文，再联合 trace 与 text 生成根因解释。

TELLER 的教学结论：压缩不能牺牲诊断结构

0.916 / 0.900

水平/垂直视图 Step F1

83.88%

较优设置的事件压缩率

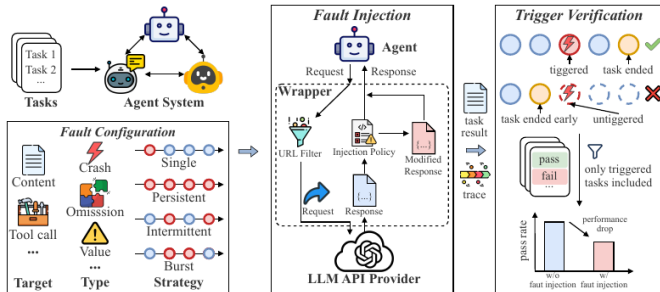
+9.8%

论文实验中的追踪墙钟开销

- ▶ trace 与 log 联合输入优于缺少任一模态；
- ▶ 更激进的压缩可达到 98.84%–99.83%，但定位 F1 明显下降；
- ▶ 因果结构对“定位在哪里”比对“粗略解释发生了什么”更加敏感。

工程取舍 可观测性数据不是越少越好，也不是越多越好；目标是在开销与诊断保真度之间找到可部署的平衡。

AgentChaos: 把 Agent 可靠性变成可重复实验



1. 输入 Agent 系统、任务集合和故障配置；
2. 在共享的 LLM API HTTP 层拦截并修改响应，无需改动 Agent 源码；
3. 记录故障是否真正触发、执行轨迹、任务结果和性能下降。

AgentChaos: Agent 故障不只是 “API 返回 500”

维度	代表配置	对系统的影响
目标字段 故障类型 注入策略	content / tool_calls crash、omission、value single、persistent、intermittent、burst	文本错误会污染下游 Agent；工具参数错误会触发错误行动 既包含显式报错，也包含截断、空响应、编码损坏和 schema 错误 同一故障以不同节奏出现，会触发不同恢复路径与资源消耗
复合场景	API degradation、max tokens、stale data 等	模拟真实部署中多个故障共同改变响应的情况

关键设计 故障配置空间由 “目标字段 × 故障类型 × 注入策略 × 注入位置” 组成，共形成 65 种可执行配置。

AgentChaos 的结果：系统架构决定故障如何被放大

49.66%

观测到的最高总体 pass@1 降幅

0.71x-4.24x

故障导致的 LLM 调用量变化

<56%

现有规则/LLM 诊断总体准确率

- ▶ Pipeline 架构会把早期故障传给所有下游阶段，首阶段注入降幅最高达 83.87%；
- ▶ 复合故障的降幅最高达 86.36%，persistent 注入最高达 62.39%；
- ▶ 有些系统不是报错退出，而是返回“看似合理但错误”的结果；论文中静默失败占比最高达 65.71%。

运维含义 只监控 API 可用性会漏掉最危险的失败：系统仍在运行、成本正在增加、答案却已经错误。

研究问题如何映射到本课程？

代表性工作	提出的系统问题	对应课程模块
CauseInfer / MicroRank / Nezha	如何利用因果图、链路和多模态证据定位根因？	异常检测、根因分析
TraStrainer / Mint / ZeroTracer	如何以可接受开销获得足够的分布式追踪证据？	可观测性工程、数据治理
LLMRCA / TELLER	如何跨越引擎、框架、CUDA 和 GPU 做 LLM 系统诊断？	AI 推理可观测性、RCA Agent
AgentChaos	如何系统验证 Agent 在 API 与响应故障下的鲁棒性？	AgentOps、安全与综合实践

研究生视角 读论文时同时追问：系统假设是什么？证据从哪里来？开销如何控制？结果能否转化为可执行的运维动作？

六、课堂互动：把架构画出来

用一个简化任务，练习从业务目标推导系统边界和观测证据。

小组任务：设计一个企业运维助手

业务需求

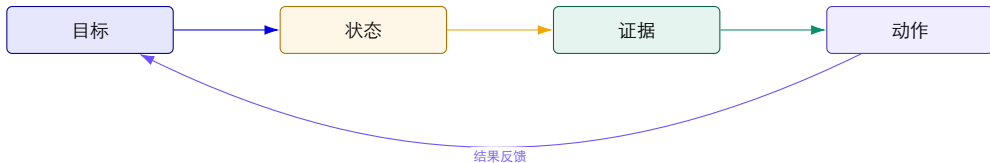
- ▶ 运维人员输入：“订单服务昨晚为什么变慢？”
- ▶ 系统需要查询指标、日志、链路、发布和对象档案；
- ▶ 输出诊断依据、根因候选和建议动作；
- ▶ 危险动作必须经过人工确认。

小组产出

1. 一张端到端架构图；
2. 三个工具定义；
3. 五个必须记录的证据字段；
4. 一个需要人工确认的动作。

时间 8 分钟小组讨论 + 2 分钟汇报。

任务提示：先写运行状态，再画组件



目标 解释“变慢”是服务问题还是业务问题。

状态 服务、Pod、接口、模型、GPU、发布。

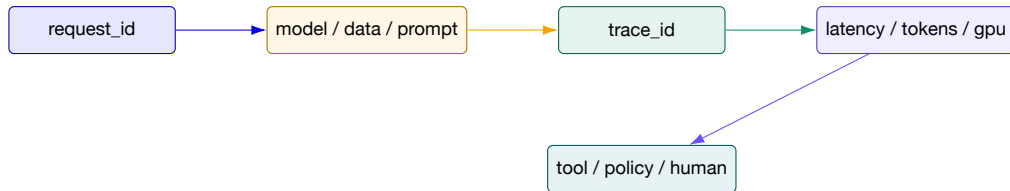
证据 时间、请求 ID、版本、指标、日志、链路。

汇报评价量表

维度	好的答案应该体现	常见问题
边界	清楚区分 Agent、数据平台、模型服务和运维平台	把所有能力都塞进一个“大模型”
证据	每个结论都有可查询的指标、日志或链路	只说“模型判断是 GPU 问题”
安全	高风险动作需要权限和人工确认	让 Agent 直接执行重启/回滚
可恢复	支持超时、降级、回滚、重放和复盘	只设计成功路径
可演进	能替换模型、知识库和工具而不破坏平台	组件强耦合、版本不可追踪

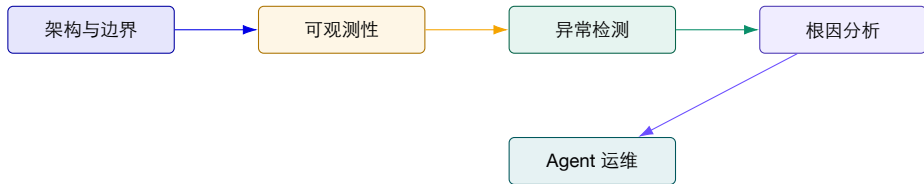
教师追问 你的系统如何证明自己判断正确？如果判断错了，谁能发现、谁能回滚？

课堂练习：给请求加上“可诊断性”



“不可观测的智能，无法稳定地进入生产。”

把今天的架构与后续课程连接起来



本讲 定义对象、状态和边界。

下一步 建立 Metrics / Logs / Traces 数据采集与存储。

最终目标 用运行数据核对诊断结论，并检查处置是否有效。

七、总结与下一讲

今天学习系统组成、运行状态，以及怎样判断一次任务是否完成。

容量算例：KV Cache 如何把输入长度变成并发限制？

简化模型：32 层、8 个 KV 头、每头维度 128，K/V 都用 2 字节；可用 KV 空间为 8 GiB。

项目	计算	结果	假设/限制
每 Token KV	$2 \times 32 \times 8 \times 128 \times 2$	131,072 字节	首个 2 表示 K 和 V
4,096 Token	$4096 \times 128 \text{ KiB}$	512 MiB / 请求	包含已缓存的序列位置
8,192 Token	$8192 \times 128 \text{ KiB}$	1 GiB / 请求	输入与已生成 Token 共同占用
理想并发上限	8 GiB / 单请求 KV	16 或 8 个请求	暂不计碎片、共享与预留

输入增长会挤占其他请求的 KV 空间，进而改变准入和排队。

推导与判断 上述结果是内存容量上限，不是吞吐预测；权重、临时张量和系统预留必须先从总显存中扣除。

算例使用假设数据。

课后思考题

1. 选择一个你熟悉的 AI 应用，画出其训练、推理、数据和运维边界；
2. 指出其中三个“模型输出正确但系统仍然失败”的场景；
3. 为一次请求设计最小可诊断字段集合；
4. 如果只能新增一个指标，你会选择 TTFT、GPU 显存、召回相关性还是工具成功率？为什么？

下次课将进入 AI 原生系统的运行挑战：推理超时、显存 OOM、模型漂移、幻觉与 Prompt 注入。

本讲参考材料

- ▶ 课程大纲：《中山大学研究生课程教学大纲-课程表修订版-20260704》；
- ▶ 训练：《Distributed Model Training》《HLAT》及 ai-infra mod-107；
- ▶ 推理：《LLM Inference》《PagedAttention》《Prefill-Decode Disaggregation》与 llm-d；
- ▶ RAG：《METIS》及 ai-infra mod-110 RAG Systems；
- ▶ Agent：《AI Agents Overview》《Augmented LLM Serving》《Agents for Enterprise Workflows》；
- ▶ Agent Context & Memory、Prefix Caching、Speculative Decoding 相关材料；
- ▶ 《AgentOps and Agent Security》；
- ▶ AI 系统故障综述、TELLER、AgentChaos 等论文；
- ▶ 'ai-infra-engineer-learning-main' 中 GPU、LLM 平台、RAG 和可观测性模块。

引用说明 论文信息已与作者公开论文列表、论文 PDF 和公开元数据交叉核验；外部课件只抽取独立图片或内容，论文原图逐页标注出处。

谢谢 | 欢迎讨论

面向AI原生系统的智能运维
校企联合研究生课程